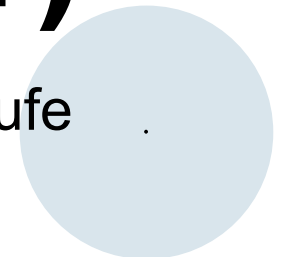


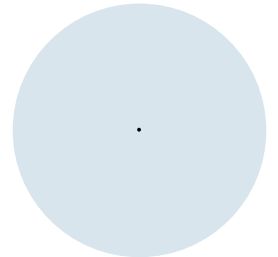
Einführung in die Programmierung (WIN+BIT)

Speichermodell, Initiale Attributwerte, NullPointer, Methodenaufrufe

Prof. Dr. Johannes C. Schneider



**Initiale Attribute-Werte,
Null-Pointer,
Objekte im Speicher**



Initialer Wert eines Klassenattributs

```
public class Address {  
    public String name;  
    public String street;  
    public String city;  
    public int postcode;  
    public boolean favorite;  
}
```

```
Address address = new Address();  
System.out.println(address.favorite); //false  
System.out.println(address.postcode); //0  
System.out.println(address.name); //null
```

Standardwerte nach Initialisierung:

- für Zahlen 0,
- für Variablen vom Typ boolean den Wert false,
- für char das Unicode-Zeichen \u0000 und
- für Referenzvariablen **null**.

Gilt übrigens
genauso für die
initialen Werte
von Arrays!

Objekte im Speicher

Code

Stack Memory

Heap Space

```
Address address;
```

- Variable ist deklariert, aber nicht initialisiert.
- Damit hat sie noch gar keinen Wert zugeordnet bekommen (weder im Stack noch im Heap)

Objekte im Speicher

Code

```
Address address;
```

```
address = null;
```

Stack Memory

```
address: null
```

Heap Space

- Variable ist mit null initialisiert
- Damit hat sie im Stack den Wert null, aber noch keinen Speicher im Heap zugeordnet.

Objekte im Speicher

Code

```
Address address;
```

```
address = null;
```

```
address = new Address();
```

Stack Memory

address: ~~null~~ 00AB

Heap Space

Hier einfache Variante von Address
nur mit city und postcode

00AB:

null	city
0	postcode

- Mit new wurde eine Instanz der Klasse Address erzeugt.
- Damit wird Speicher im Heap reserviert mit Platz für eine String-Referenz city und einen int-Wert postcode (primitiver Datentyp)
- address zeigt im Stack auf Speicheradresse (Referenz) im Heap
- nach Initialisierung haben city und postcode die default-Werte

Objekte im Speicher

Code

```
Address address;
```

```
address = null;
```

```
address = new Address();  
[ address.city  
  = "Konstanz";  
  address.postcode = 78464;
```

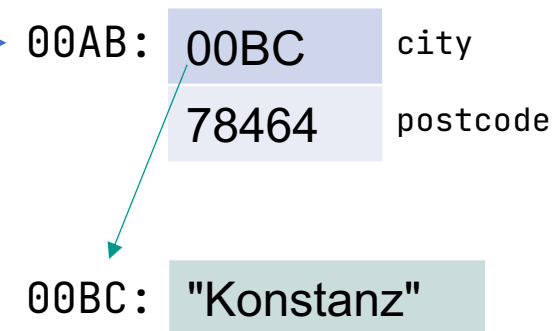
Stack Memory

address: null 00AB

- Referenz auf Address-Objekt
- Referenz auf String-Objekt

Heap Space

Hier einfache Variante von Address
nur mit city und postcode



- Mit den beiden Zuweisungen werden die Attribute city und postcode im Heap neu gesetzt:
 - city zeigt auf den neu im Heap angelegten String "Konstanz"
 - postcode wird direkt im Speicherbereich der Adressinstanz geändert (primitiver Datentyp)

Null-Pointer: Fehlergefahr!

```
String string = null;
```

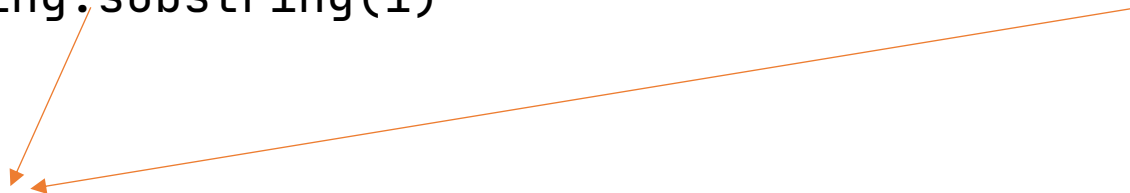
- string ist noch nicht initialisiertes Objekt vom Typ String

```
Address address = null;
```

- address ist noch nicht initialisiertes Objekt vom Typ Address

```
– string.substring(1)
```

```
address.name
```



- Fehler zur Laufzeit (*NullPointerException*)
 - `null.substring(1)` kann nicht aufgerufen werden
 - `null.name` kann nicht aufgerufen werden
- Eine der häufigsten Fehlerursachen bei Java-Programmen!



code14.
NullPointerException

NullPointerException

- NullPointerException sind eine der häufigsten Fehlerursachen bei Java-Programmen!

```
Address beate = new Address();
beate.name = "Beate Beispiel";
beate.street = "Lindenstr. 11";
beate.postcode = 78464;

Address max = new Address();
max.name = "Max Muster";
max.street = "Lindenstr. 12";
max.postcode = 78464;
max.city = "Konstanz";

printNeighbourhoudInfo(beate, max);
```

```
private static void printNeighbourhoudInfo(
    Address a1, Address a2) {
    if (a1.city.equals(a2.city)) {
        if (a1.street.equals(a2.street)) {
            System.out.println(a1.name + " u
        } else {
            System.out.println(a1.name + " u
        }
    } else {
        System.out.println(a1.name + " und "
    }
}
```

Exception in thread "main" java.lang.NullPointerException

→ Warum?



code14.

NullPointerException

NullPointerException verhindern!

- Vor Zugriff auf eine Variable, die null sein könnte, überprüfen, ob diese null ist:

```
if (a1.city != null) {  
    if (a1.city.equals(a2.city)) {  
        // ...  
    }  
} else {  
    // ...  
}
```

← An dieser Stelle ist klar, dass `a1.city` nicht null ist. Man kann also auf `a1.city` zugreifen.

- oder kürzer:

```
if (a1.city != null && a1.city.equals(a2.city)) {  
    // ...  
} else {  
    // ...  
}
```



code14.
NullPointerException

NullPointerException verhindern!

```
if (a1.city != null && a1.city.equals(a2.city)) {  
    // ...  
} else {  
    // ...  
}
```

- Dieser Code kann immer noch NullPointerExceptions werfen! Wann?
- Jetzt werden alle NullPointerExceptions vermieden:

```
if (a1 != null && a1.city != null &&  
    a2 != null && a1.city.equals(a2.city)) {  
    // ...  
} else {  
    // ...  
}
```



code14.
NullPointerException

Erinnerung: Short-Circuit-Auswertung

```
if (a1 != null && a1.city != null /* ... */) {  
    // ...  
}
```

- Wenn **linker Teil** des &&-Operators schon false, also `a1 == null` gilt, wird der rechte Teil nicht ausgewertet
- Es wird dann also nicht auf `a1.city` zugegriffen
- Hier kommt es also zu keiner `NullPointerException` bei `a1.city`



code14.
NullPointerException

Objekte im Speicher, Wiederholung

Code

```
Address address =  
    new Address();  
address.city  
    = "Konstanz";  
address.postcode = 78464;
```

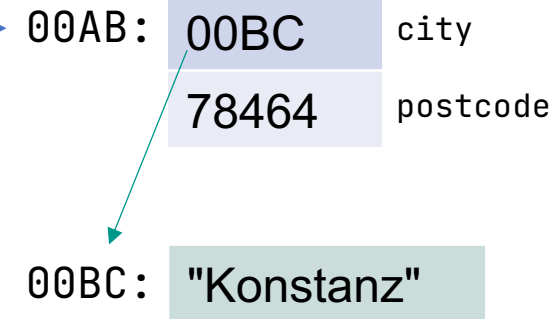
Hier wieder einfache Variante von
Address nur mit city und
postcode

Stack Memory

address: 00AB

- Referenz auf Address-Objekt
- Referenz auf String-Objekt

Heap Space



- Variablen von Referenztypen bekommen über die Objekterzeugung mit new eine Speicheradresse im Heap zugewiesen

Im Beispiel:

- address zeigt im Stack auf Speicheradresse (Referenz) im Heap
- Address-Objekt im Heap besteht aus Referenz auf city (String) und aus dem postcode (Wert als elementarer Typ direkt im Objekt)

Variablenkopie

Code

```
Address address =  
    new Address();  
address.city  
    = "Konstanz";  
address.postcode = 78464;
```

```
Address another = address;
```

Stack Memory

address: 00AB

another: 00AB

Zuweisung erzeugt Kopie
des Werts im Stack

Heap Space

00AB: 00BC city
78464 postcode

00BC: "Konstanz"

→ Referenz auf Address-Objekt
→ Referenz auf String-Objekt

- another zeigt auf gleichen Bereich im Heap wie address. Also erreicht another.city und another.postcode dieselben Werte wie address.city und address.postcode.



code14.
AddressIn
Memory

Wiederholung: Address-Instanz im Array

```
final int MAX_ADDRESSES = 20;
Address[] addresses = new Address[MAX_ADDRESSES];
for (int i = 0; i < addresses.length; i++) {
    addresses[i] = new Address();
}
addresses[0].name = "Beate Beispiel";
addresses[0].street = "Lindenstr. 11";
addresses[0].city = "Konstanz";
addresses[0].postcode = 78464;
addresses[0].favorite = false;

for (Address address : addresses) {
    System.out.println(address.name);
    // ...
}
```



code14.
MainArray

Address-Instanz im Array, Zuweisung

```
final int MAX_ADDRESSES = 20;
Address[] addresses = new Address[MAX_ADDRESSES];
:
Address max = new Address();
max.name = "Max Muster";
max.street = "Lindenstr. 13";
max.city = "Radolfzell";
max.postcode = 78464;
max.favorite = true;

addresses[1] = max;
System.out.println(addresses[1].city); // Ausgabe: Radolfzell
```

- max wird im Array der Platz 1 zugewiesen und später werden alle Werte ausgegeben.
- Warum funktioniert das?



code14.
MainArray

Address-Instanz und Arrays

Code

```
Address max =  
    new Address();  
address.city  
    = "Konstanz";  
address.postcode = 78464;
```

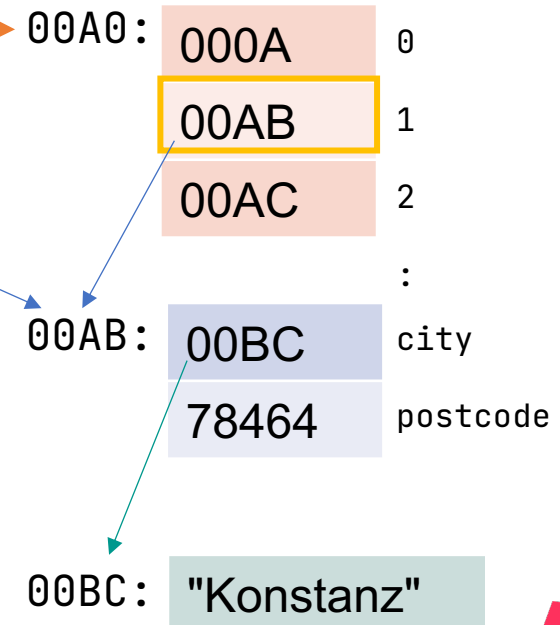
```
addresses[1] = max;
```

Stack Memory

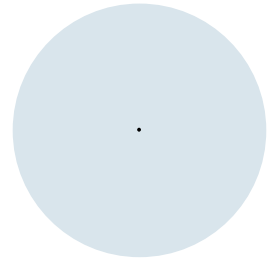
```
addresses: 00A0  
max: 00AB
```

- Referenz auf Array
- Referenz auf Address-Objekt
- Referenz auf String-Objekt

Heap Space



Methodenaufrufe, Stack versus Heap



Wirkung von Methodenaufrufen

Methodenaufrufe wie

```
doSomething(argument);
```

- Auf welche Speicherbereiche haben sie Einfluss?
- Wie können sie argument verändern, so dass die Änderungen auch außerhalb der Methode sichtbar sind?

Hinweis zu den Beispielen

- Es gibt für die folgenden Beispiele auch eine Variante `<Beispiel>MorePrintln`
 - `ParameterPassingInt -> ParameterPassingIntMorePrintln`
 - ...
- Diese enthalten mehr `System.out.println`-Ausgaben (ggf. leichter zu verstehen für die Nachbereitung)

Aufruf mit primitivem Parameter

```
public static void doSomething(int intParam) {  
    intParam = 11;  
}  
  
public static void main(String[] args) {  
    int intMain = 42;  
    doSomething(intMain);  
    System.out.println(intMain);  
}
```



code14.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;
```

Stack Memory

```
intMain: 42
```

Heap Space

– Wert im Stack



code14.
Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain) {
```

Stack Memory

```
intMain: 42  
intParam: 42
```



Heap Space

- Für den Aufruf wird der Inhalt von intMain im Stack nach intParam kopiert.



code14.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain) {  
    intParam = 11;  
}
```

Stack Memory

```
intMain: 42  
intParam: 11
```

Heap Space

- innerhalb doSomething wird intParam im Stack geändert



code14.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain);  
System.out  
    .println(intMain);
```

Stack Memory

intMain: 42

Heap Space

- zurück in der Main-Methode liefert intMain immer noch den Wert 42.
- der Stack-Teil mit intParam hat seine Gültigkeit verloren (Werte werden wieder aus dem Stack entfernt)



code14.

Parameter
PassingInt

Klasse Person

```
public class Person {  
    public String name;  
    public int age;  
}
```

Aufruf mit Referenztyp-Parameter

```
public static void doSomething(Person personParam) {  
    personParam = new Person();  
    personParam.name = "Pia";  
    personParam.age = 22;  
}  
  
public static void main(String[] args) {  
    Person personMain = new Person();  
    personMain.name = "Paul";  
    personMain.age = 42;  
  
    doSomething(personMain);  
    System.out.println(personMain.name + ", " + personMain.age);  
}
```



code14.

Parameter
PassingObject1

Aufruf mit Referenztyp-Parameter

Code

```
Person personMain =  
    new Person();  
personMain.name = "Paul"  
personMain.age = 42;
```

Stack Memory

personMain: 00AB

Heap Space

00AB: 00BC

42

00BC: "Paul"

- Referenz auf Person-Objekt
- Referenz auf String-Objekt

- personMain zeigt im Stack auf Speicheradresse (Referenz) im Heap
- Person-Objekt im Heap besteht aus Referenz auf den Namen (String) und dem Alter (Wert als elementarer Typ direkt im Objekt)



code14.

Parameter
PassingObject1

Aufruf mit Referenztyp-Parameter

Code

```
Person personMain =  
    new Person();  
personMain.name = "Paul"  
personMain.age = 42;  
doSomething(personMain) {
```

Stack Memory

personMain: 00AB
personParam: 00AB

Kopie

Heap Space

00AB: 00BC
42
00BC: "Paul"

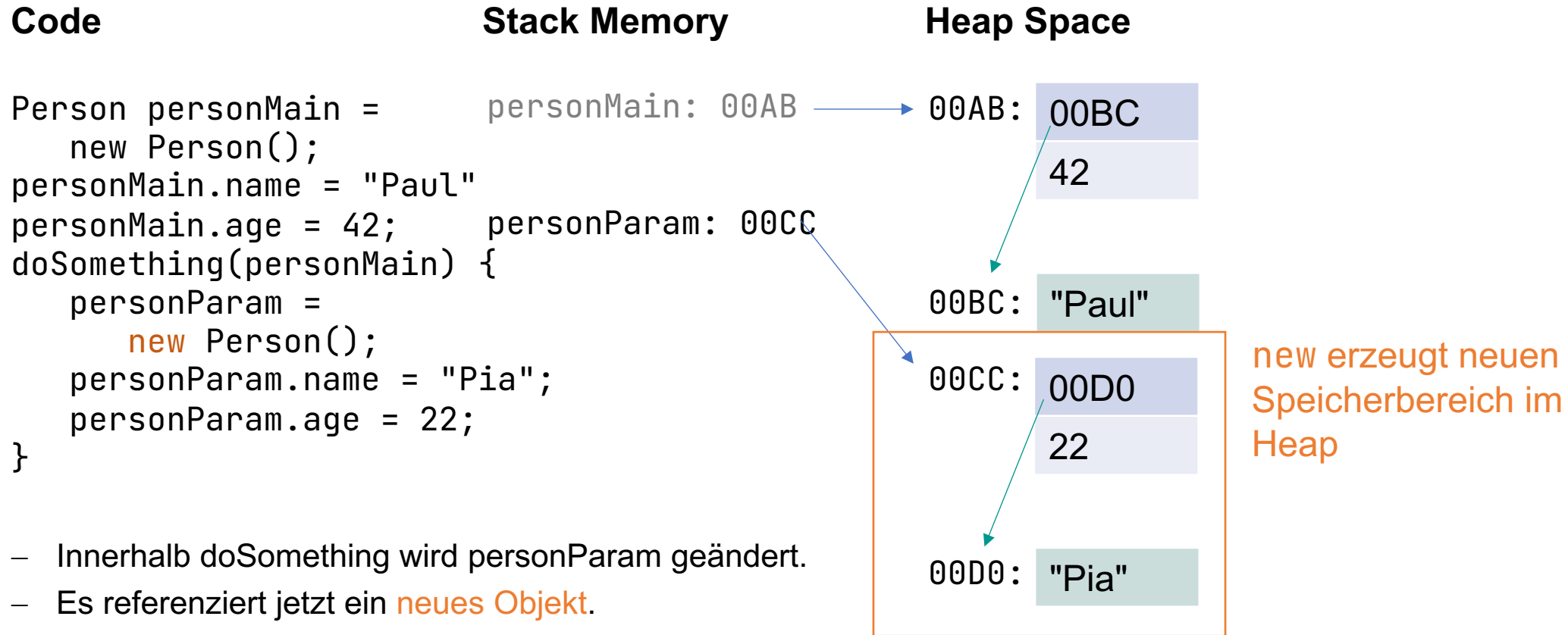
- Für den Aufruf wird der Wert von personMain im Stack nach personParam kopiert.



code14.

Parameter
PassingObject1

Aufruf mit Referenztyp-Parameter



- Innerhalb doSomething wird personParam geändert.
- Es referenziert jetzt ein **neues Objekt**.
- D.h. im Stack steht bei personParam eine neue Speicheradresse.



code14.
Parameter
PassingObject1

Aufruf mit Referenztyp-Parameter

Code

```
Person personMain =  
    new Person();  
personMain.name = "Paul";  
personMain.age = 42;  
doSomething(personMain);  
System.out  
    .println(personMain.name);  
System.out  
    .println(personMain.age);
```

Stack Memory

personMain: 00AB

Heap Space

00AB: 00BC

42

00BC: "Paul"

00CC: 00D0

22

00D0: "Pia"

- zurück in der Main-Methode liefert personMain immer noch dieselbe Person.
- der Stack-Teil mit personParam hat seine Gültigkeit verloren (Werte werden wieder aus dem Stack entfernt)
- der Garbage-Collector löscht irgendwann die nirgendwo mehr referenzierte Person Pia



code14.

Parameter
PassingObject1

Parameterübergabe

Call-by-value:

- Parameter der Methode wird mit Kopie des Variablen-**Werts** belegt
- Wert kann Größe ("Skalar") sein (primitiver Datentyp) oder Speicheradresse (Referenz-Datentyp)

Call-by-reference:

- Parameter ist **Referenz** auf die ursprüngliche Variable
- Änderungen an der Parameter-Referenz ändern auch die Referenz der ursprünglichen Variable

Parameterübergabe

- Entgegen vieler Lehrwerke (auch D. Abts) gibt es **bei Java** kein **"Call-by-reference"**, da die Referenz beim Aufrufer nicht geändert wird (personMain zeigt nach wie vor auf dasselbe Objekt)
 - In anderen Programmiersprachen wie C++ ist es möglich, auch die personMain-Referenz zu ändern
- Es ist korrekterweise ein *"Call-by-Value where the value is a reference"*
- Trotzdem ist es natürlich möglich, den **Inhalt** des referenzierten Objekts zu ändern. Das zeigen wir jetzt!

Inhaltsänderung des referenzierten Objekts

```
public static void doSomething(Person personParam) {  
    personParam.name = "Pia";  
    personParam.age = 22;  
}  
  
public static void main(String[] args) {  
    Person personMain = new Person();  
    personMain.name = "Paul";  
    personMain.age = 42;  
  
    doSomething(personMain);  
    System.out.println(personMain.name + ", "  
                        + personMain.age);  
}
```

Hier wird auf den über
personParam referenzierten
Speicherbereich zugegriffen.



code14.

Parameter
PassingObject2

Inhaltsänderung des referenzierten Objekts

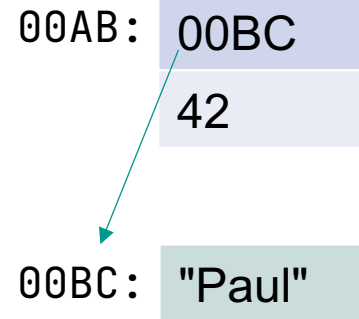
Code

```
Person personMain =  
    new Person();  
person.name = "Paul"  
person.age = 42;
```

Stack Memory

personMain: 00AB

Heap Space



- Referenz auf Person-Objekt
- Referenz auf String-Objekt

- personMain zeigt im Stack auf Speicheradresse (Referenz) im Heap
- Person-Objekt im Heap besteht aus Referenz auf den Namen (String) und dem Alter (Wert als elementarer Typ direkt im Objekt)



code14.

Parameter
PassingObject2

Inhaltsänderung des referenzierten Objekts

Code

```
Person personMain =  
    new Person();  
person.name = "Paul"  
person.age = 42;  
doSomething(personMain) {
```

Stack Memory

personMain: 00AB
personParam: 00AB

Kopie

Heap Space

00AB: 00BC
42
00BC: "Paul"

- Für den Aufruf wird der Wert von personMain im Stack nach personParam kopiert.



code14.

Parameter
PassingObject2

Inhaltsänderung des referenzierten Objekts

Code

```
Person personMain =  
    new Person();  
person.name = "Paul"  
person.age = 42;  
doSomething(personMain) {  
    personParam.name = "Pia";  
    personParam.age = 22;  
}
```

Stack Memory

personMain: 00AB
personParam: 00AB

Heap Space

00AB: 00CD
22
00BC: "Paul"
00CD: "Pia"

- innerhalb doSomething wird der Inhalt desjenigen Objekts geändert, zum die die Speicheradresse von personParam zeigt.



code14.

Parameter
PassingObject2

Inhaltsänderung des referenzierten Objekts

Code

```
Person personMain =  
    new Person();  
person.name = "Paul"  
person.age = 42;  
doSomething(personMain);  
System.out  
    .println(personMain.name);  
System.out  
    .println(personMain.age);
```

Stack Memory

personMain: 00AB

Heap Space

00AB: 00CD

22

00BC: "Paul"

00CD: "Pia"

- zurück in der Main-Methode liefert personMain immer noch die gleiche Referenz
- Allerdings hat die Methode den Inhalt des referenzierten Objekts geändert!
- der Stack-Teil mit personParam hat seine Gültigkeit verloren (Werte werden wieder aus dem Stack entfernt)
- der Garbage-Collector löscht irgendwann den nirgendwo mehr referenzierten String "Paul"



code14.

Parameter
PassingObject2

Inhaltsänderung Array

```
public static void doSomething(int[] arrayParam) {  
    arrayParam[1] = 11;  
}  
  
public static void main(String[] args) {  
    int[] arrayMain = new int[]{42, 43};  
    doSomething(arrayMain);  
    System.out.println(Arrays.toString(arrayMain));  
}
```

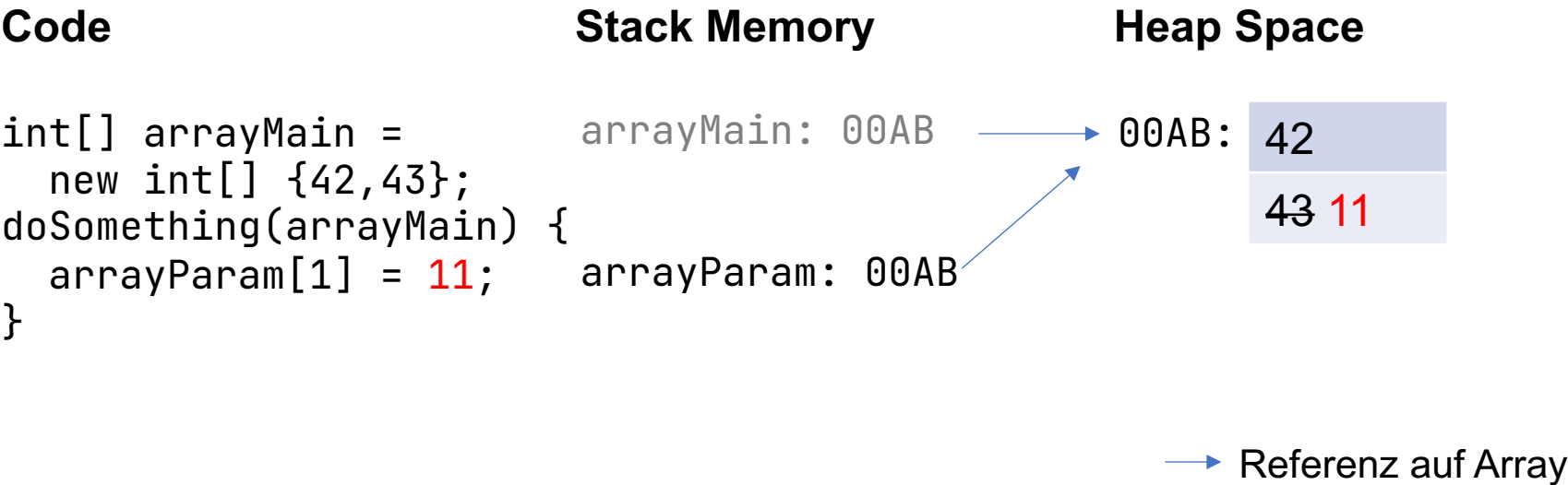
Hier wird auf den über
arrayParam referenzierten
Speicherbereich zugegriffen.



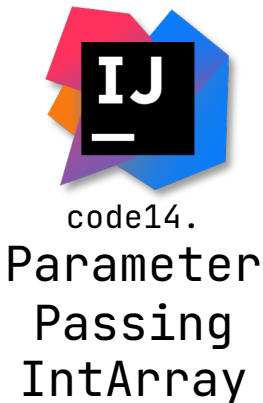
code14.

Parameter
Passing
IntArray

Inhaltsänderung Array



- Inhaltsänderung des referenzierten Objekts klappt auch, wenn das referenzierte Objekt ein Array ist



Genauer angesehen: String

- Strings sind Objekte (Instanzen der Klasse String)
- Aber: Strings sind **unveränderlich** (*immutable*)
 - Mit der Referenz auf einen String kann der Inhalt hinter dieser Referenz nicht geändert werden
 - Jede String-Operation wie "Hallo" + "Welt" und "BENGEL".substring(1) erzeugt eine Referenz auf einen anderen String mit dem neuen Inhalt
- D.h. in folgendem Code ist garantiert, dass am Ende hallo noch denselben Inhalt hat:

```
String hallo = "Hallo";  
doSomething(hallo); // doSomething ist beliebige  
                    // Methode mit String-Parameter  
System.out.println(hallo); // Ausgabe Hallo
```



code14.
StringsAre
Immutable



code14.
Parameter
Passing
String